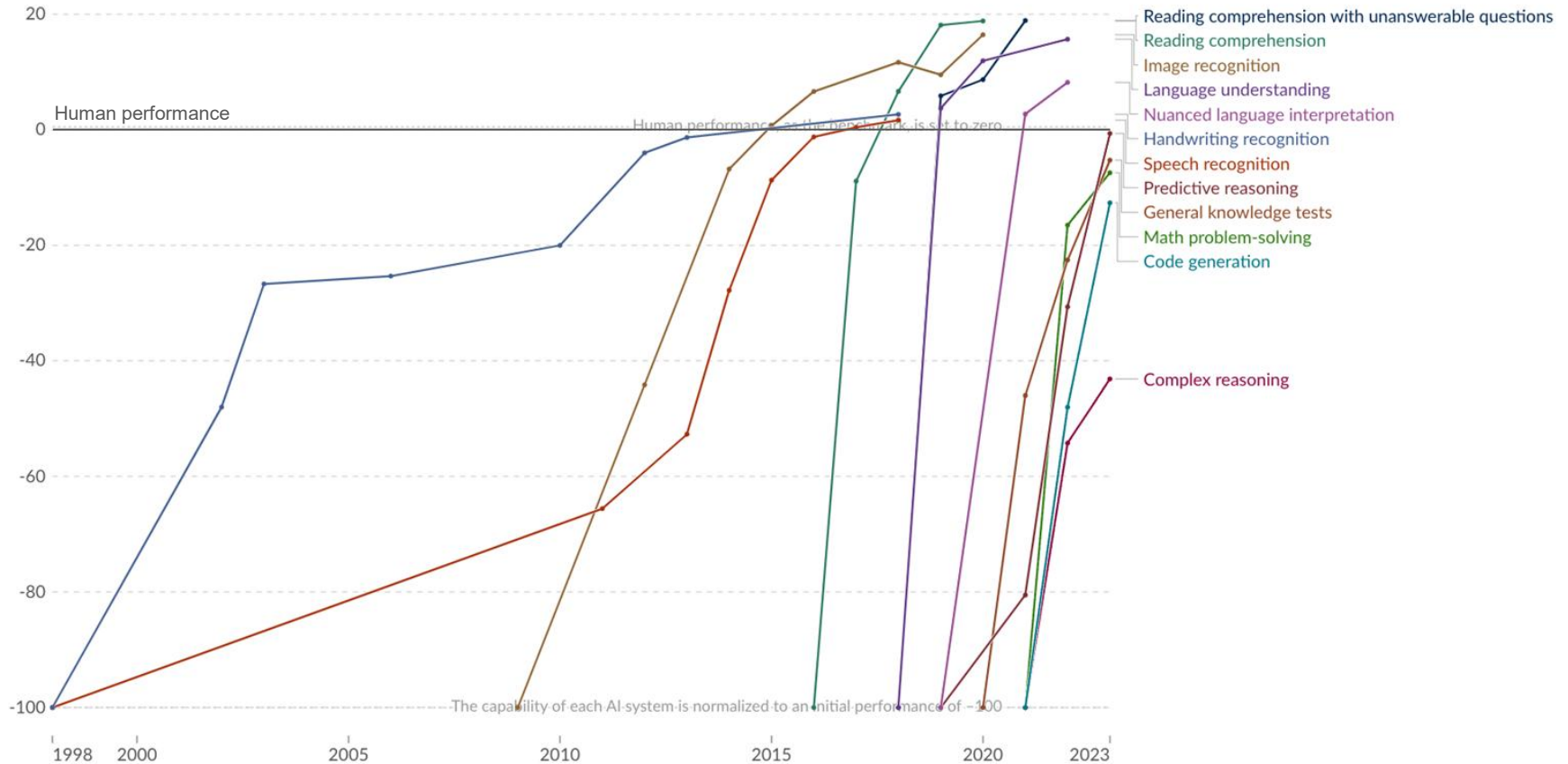


Advanced Natural Language Processing: AdvNLP

Lesson 12: Agentic AI

Prof. Dr. Jelena Milosevic (FHNW)

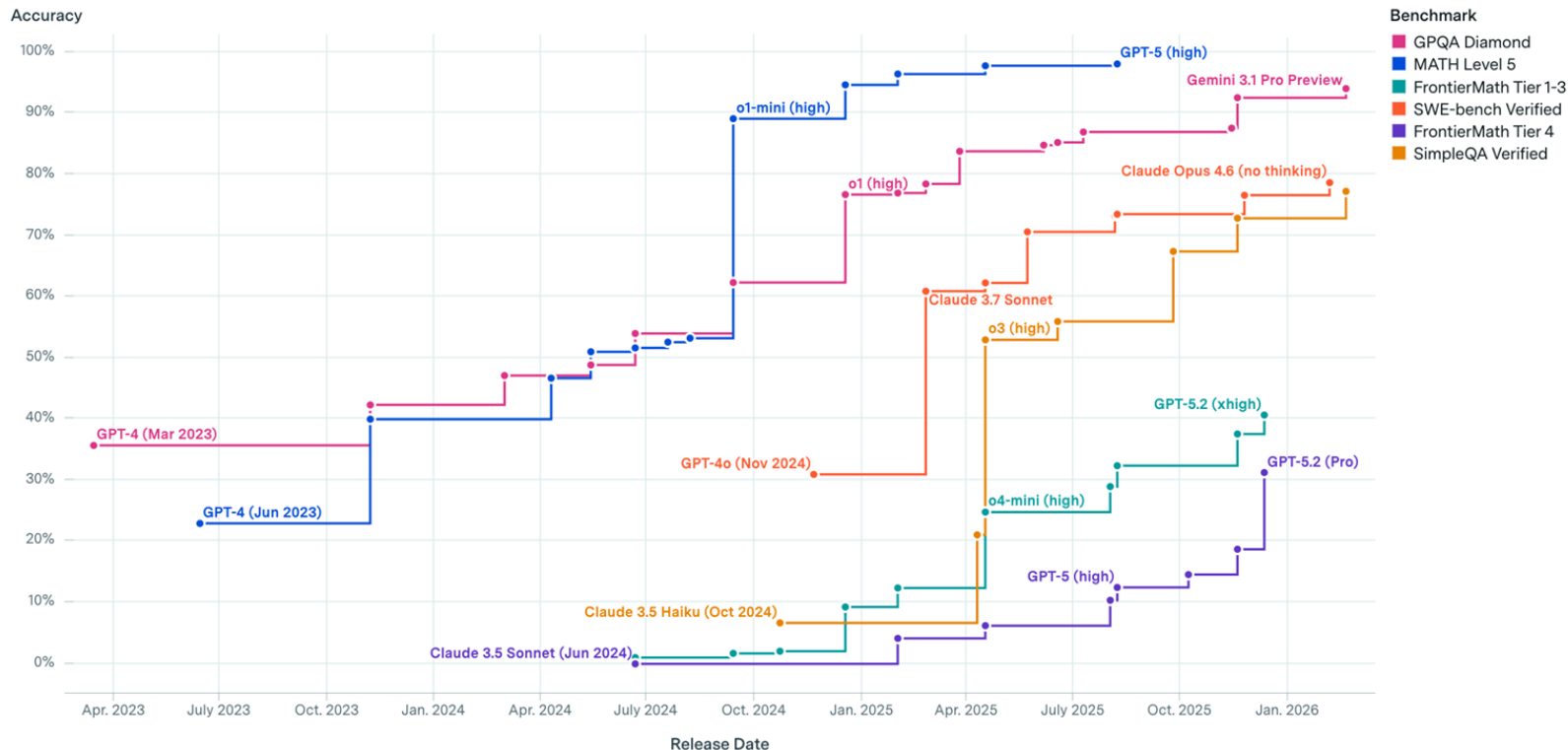
The Accelerating Advancement of AI Capabilities: Tasks



Data from Kiela et al., <https://ourworldindata.org/artificial-intelligence>

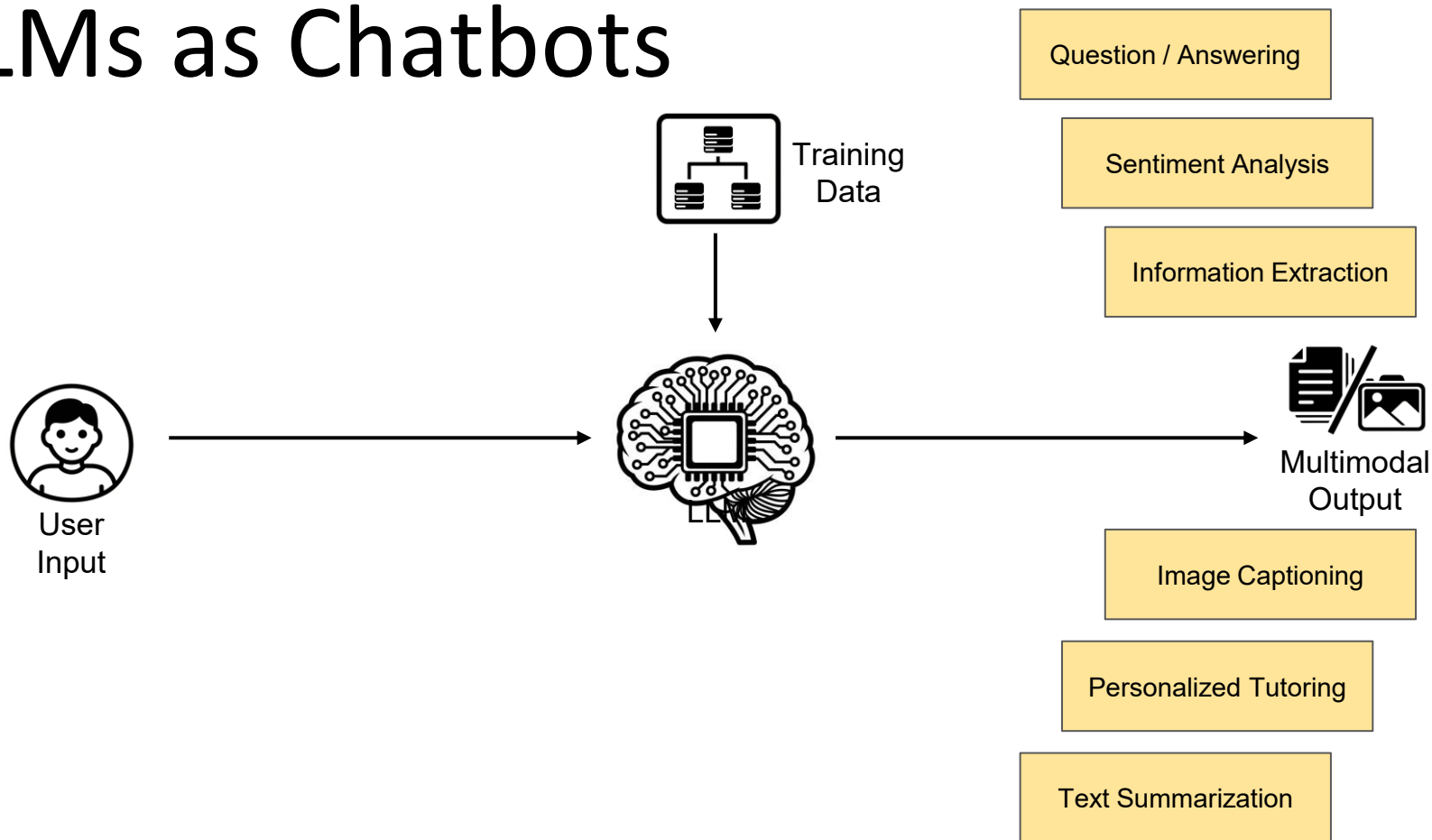
The Accelerating Advancement of AI Capabilities: Models

Frontier performance across benchmarks



Source: <https://epoch.ai/benchmarks>

LLMs as Chatbots



Challenges & Limitations

Lacks cross-session
context

Customizing
knowledge is
expensive

Static knowledge
becomes outdated

Cannot take external
actions

Struggle with complex
workflows

Human required for
every interaction

What is Agentic AI?

- **Beyond Chatbots:** Systems that don't just answer questions, but operate autonomously.
- **Core Capabilities:** They set goals, plan, and take real-world actions to achieve an outcome.
- **The difference:**
 - Traditional LLM: "List the flight options to Tokyo."
 - Agentic AI: "Book the best flight to Tokyo."

Simple Agent vs. Agentic System

- **Single AI Agent:** Operates independently to complete a specific, isolated task.
 - Example: An agent drafting a document.
- **Agentic System:** Orchestrates multiple, specialized agents working collaboratively to solve complex problems.
 - Example: A “Manager” agent delegating tasks to a “Researcher,” a “Writer,” and a “Reviewer”.

A Brief History of AI Leading to Agents



1950s–80s: Symbolic AI

The Hurdle: Brittle,
Couldn't Learn.



1980s–2010s: Machine Learning

The Hurdle:
Data-Hungry, Opaque



2010s–Present: Generative AI

The Hurdle:
Hallucinations,
Lack of Planning

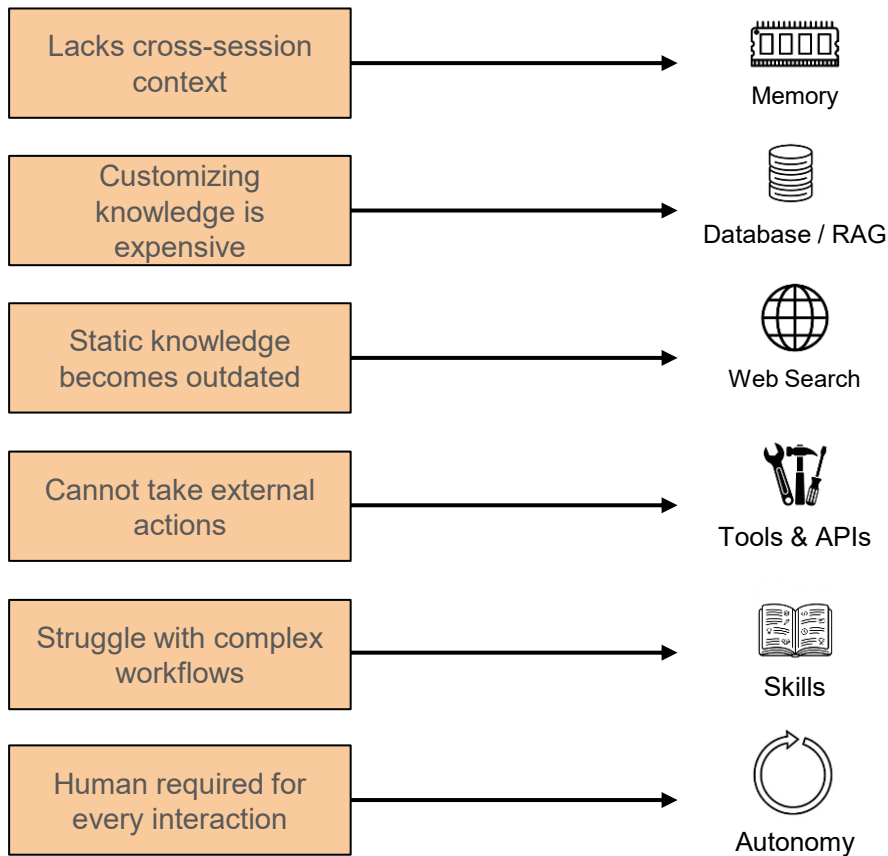


2022–Present: Agentic AI

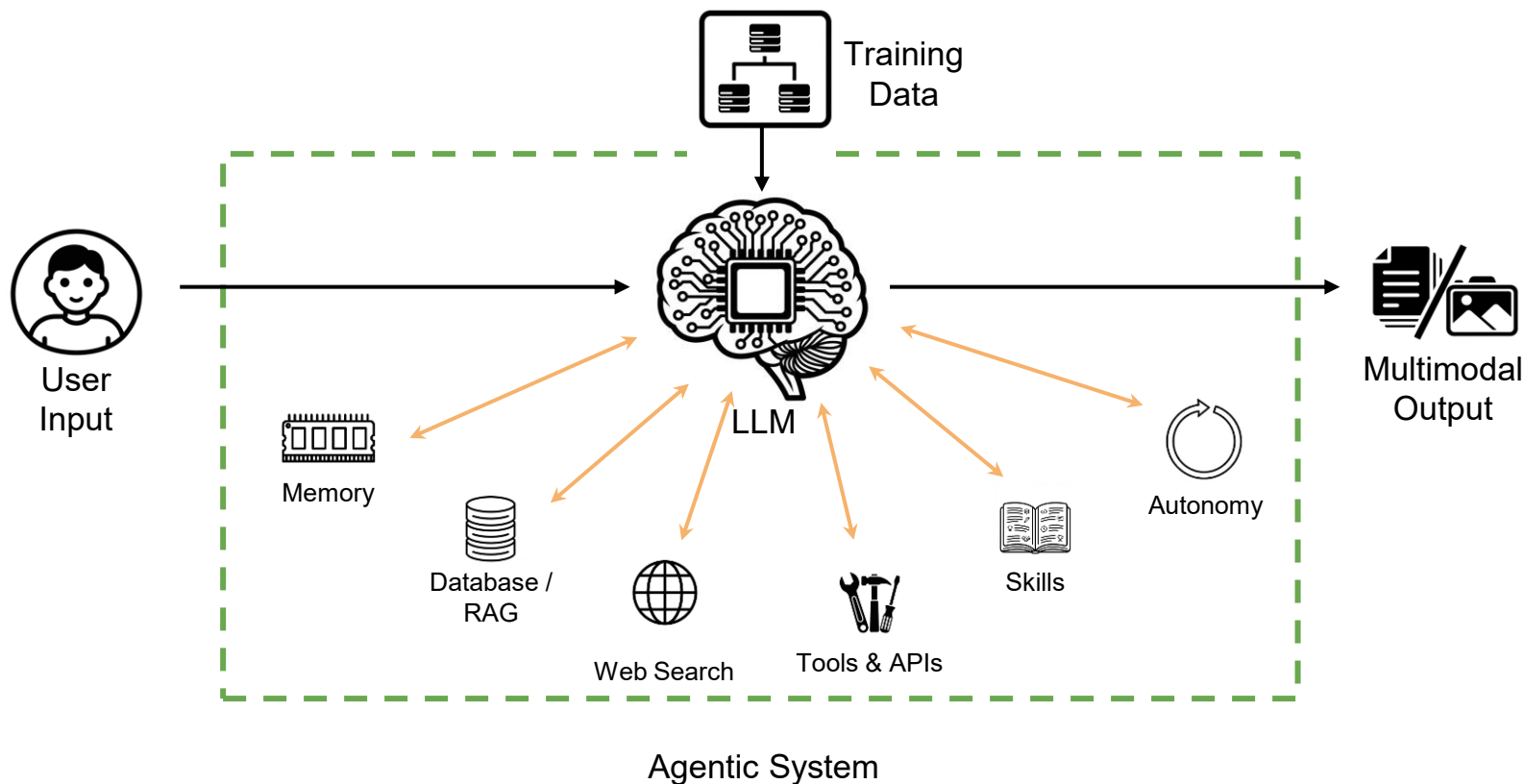
The Goal:
Autonomous
Planning & Action

Description please

Challenges & Limitations



Modern Agentic Systems Overview



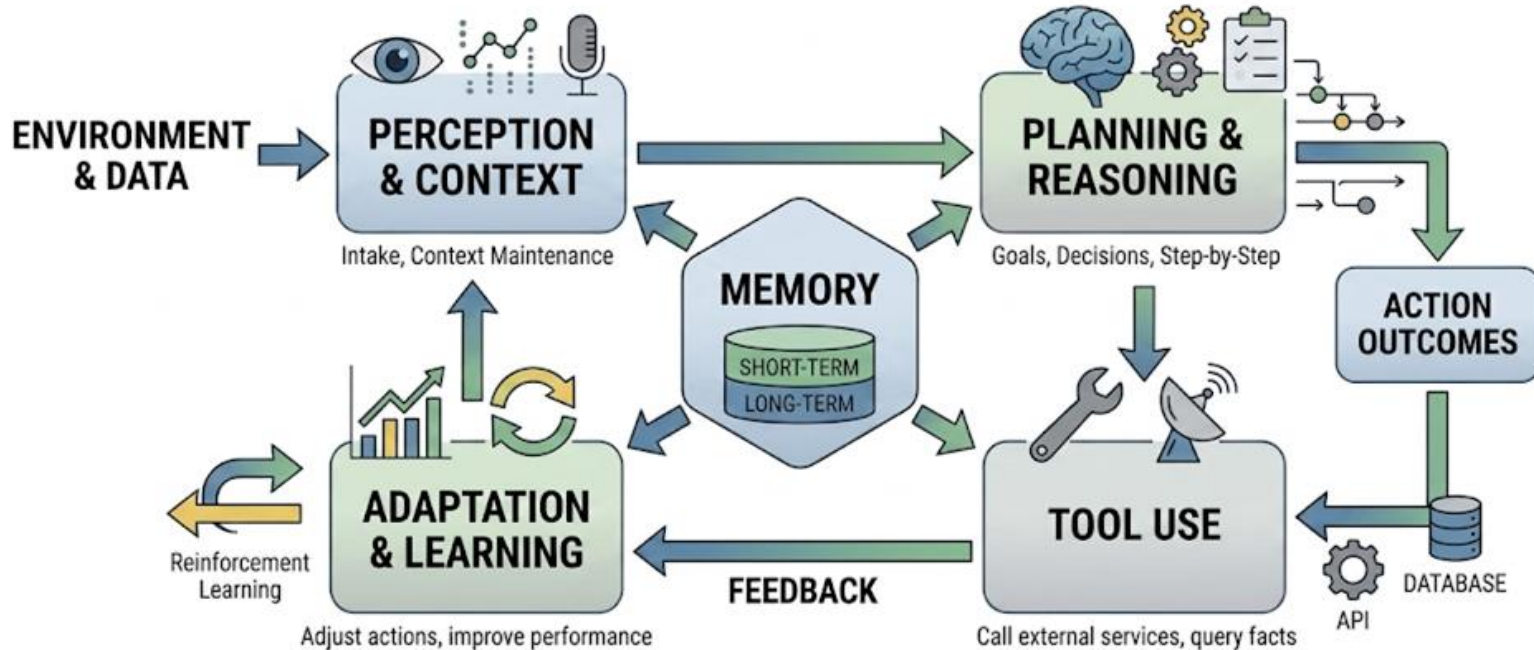
Benefits of Agentic AI

- **Massive Performance Gains:** Wrapping a basic LLM in an agentic workflow can outperform a superior, standalone model doing direct generation.
- **Parallel Execution:** Agents can run sub-tasks concurrently (e.g., executing multiple web searches at once), drastically reducing execution time.
- **Modularity:** Workflows allow developers to easily swap out underlying models or specific tools to optimize each step.

Real-World Applications

- **Software Engineering:** Coding copilots generate, test, and debug code, shifting programmers from writers to reviewers.
- **Healthcare:** Agents schedule appointments, gather patient information, and assist doctors in analyzing images for diagnosis.
- **Business Operations:** Automating invoice processing and connecting to ERP systems to resolve supply chain bottlenecks.
- **Customer Service:** Handling complex, multi-step queries like inventory checks and return policies autonomously.

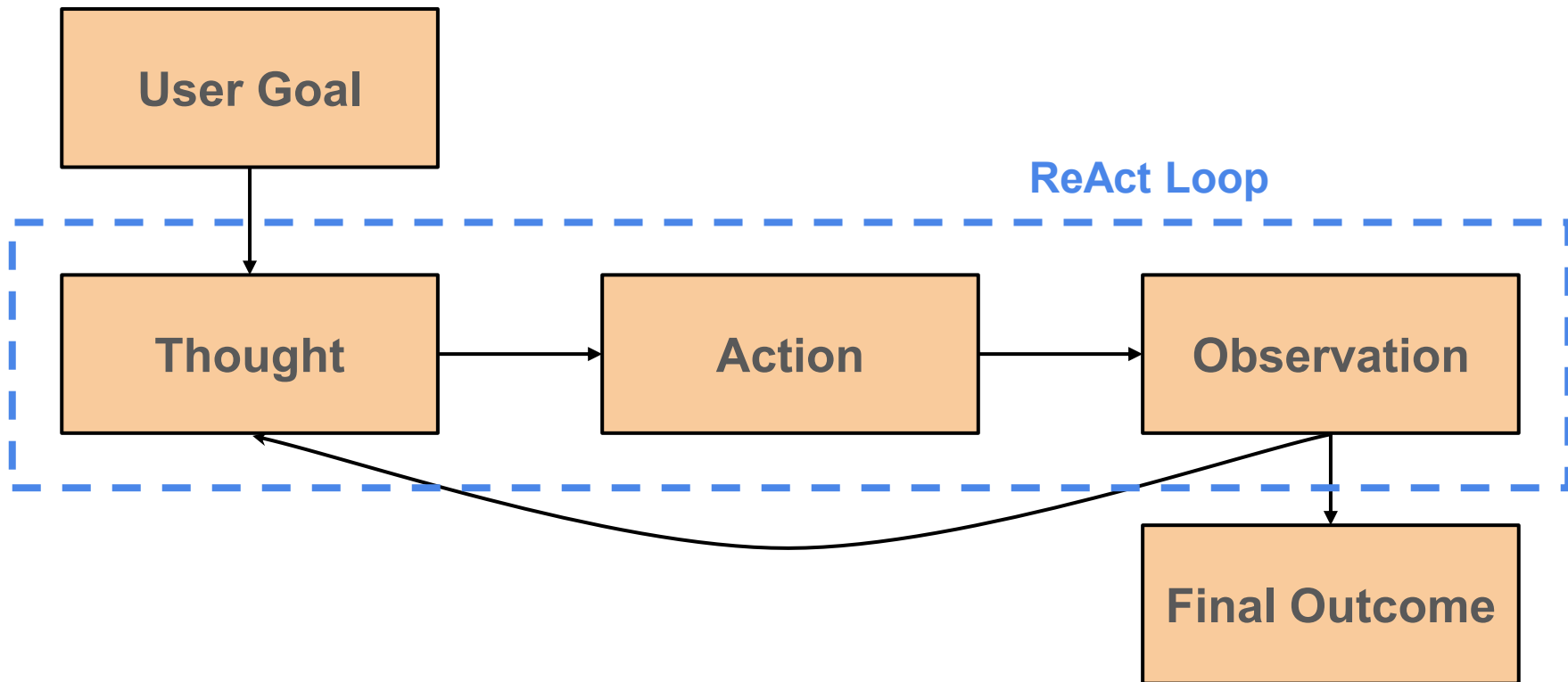
Capabilities of Agentic Systems



Perception & Context

- **Multimodal Input:** Agents don't just read text; they ingest voice, images, and structured data.
- **Active Context:** They maintain a continuous "working memory" of the ongoing conversation or task state.
- **Situational Awareness:** Understanding not just what was asked, but the environment and constraints around the request.

Planning & reasoning



Tool use

- **Beyond Text Generation:** Giving the "brain" hands to interact with the real world.
- **Common Tools:**
 - **Web Browsing:** Searching for real-time information.
 - **Code Execution:** Writing and running Python scripts to do complex math or data analysis.
 - **APIs:** Connecting to CRMs, booking systems, or enterprise databases.
- **Example:** An agent doesn't just write a SQL query; it connects to the database, runs the query, and graphs the result.

Memory

- **Short-Term Memory:** The current session's context window.
- **Long-Term Memory:** Storing facts and history across sessions (often using Vector Databases / RAG).
- **Semantic Recall:** Fetching highly specific past interactions exactly when they are relevant.
 - Still an open challenge!
- **Example:** An agent remembering a user's specific coding style or a company's strict formatting guidelines across projects happening months apart.

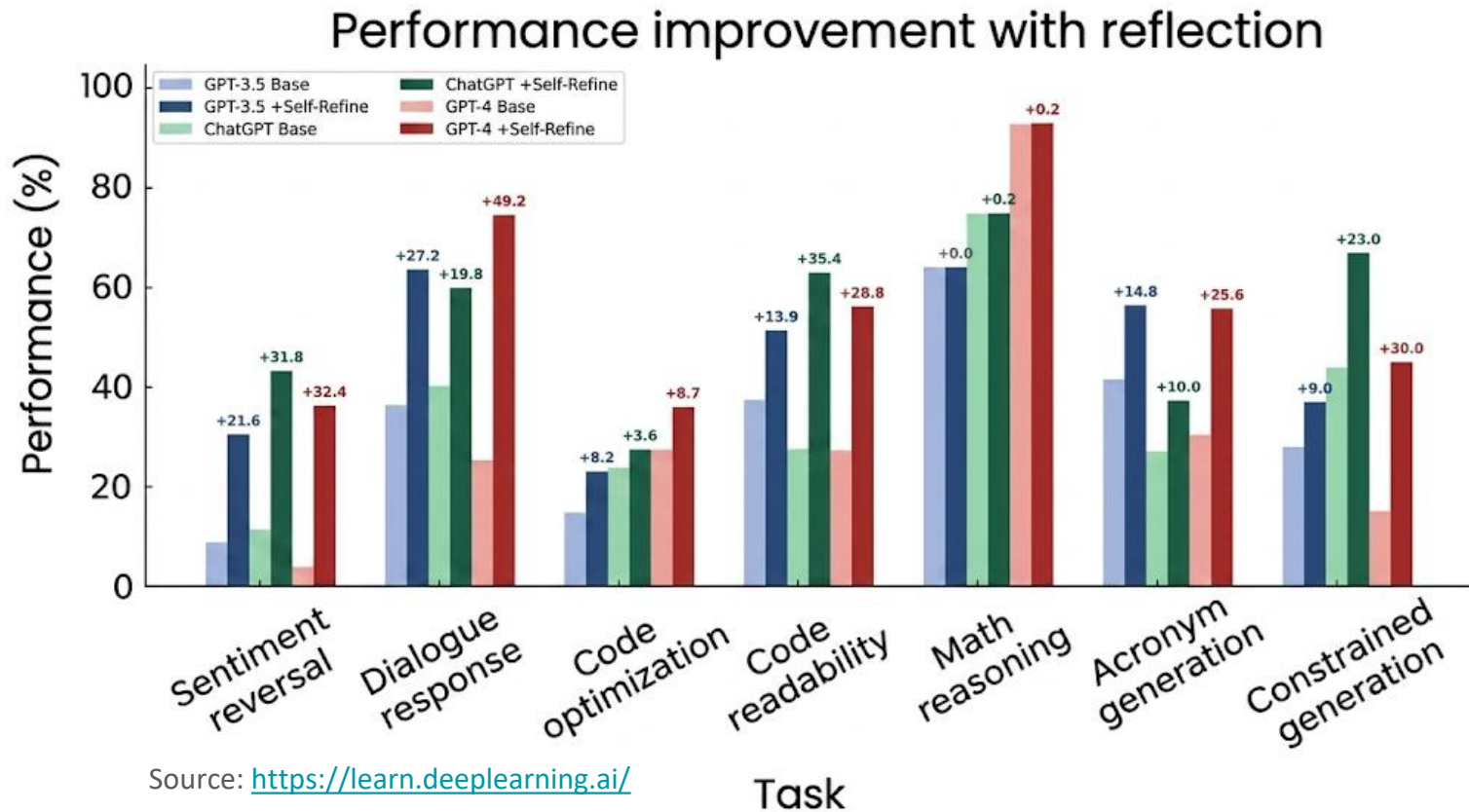
Adaptation & learning

- **Reflection:** The ability to evaluate the outcome of an action and recognize failures.
- **Self-Correction:** Adjusting the plan when an observation doesn't match the expected result.
- Different from Memory:
 - Memory is storing facts (e.g., "The API key is X").
 - Learning is an "updating strategy".

Multi-Agent Systems – Common Patterns

- **Planning (Step-by-Step Generation):** The system dynamically creates an execution plan before taking action (vs. relying on hard-coded workflows).
- **Verification & Reflection:** The agent inspects its own or peer's output. It evaluates if the task was completed successfully or if corrections are needed.
- **Complex Tool Orchestration:** Instead of a single API call, the agent strings together multiple distinct tools (e.g., extracting data, running Python to filter it, then using an API to email the result).
- **Multi-Agent Collaboration:** Decomposing complex work into specialized roles (e.g., a "Manager" delegating to "Researcher," "Designer," and "Writer" agents), similar to a human team structure.

Reflection



Planning Example: Customer Service Agent

Inventory Database

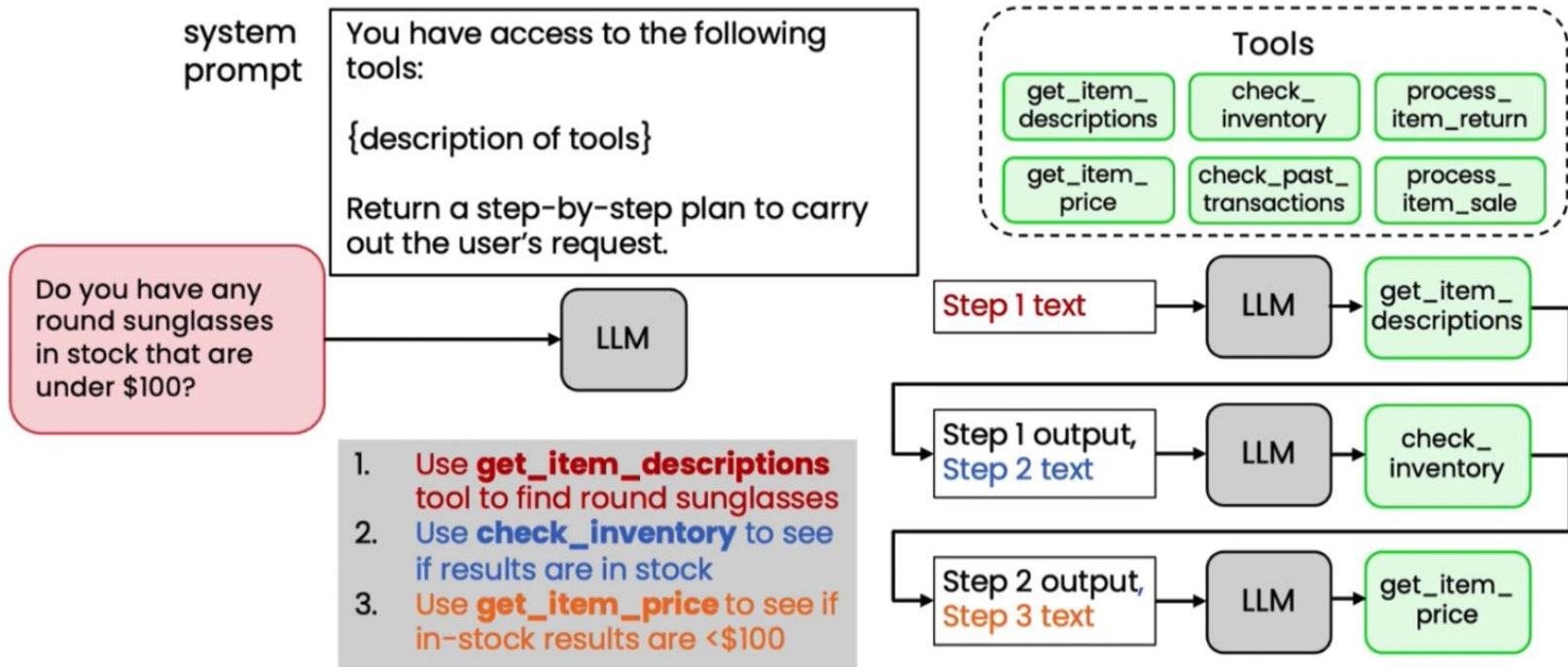
id	name	description	price	stock
1001	Aviator	Timeless pilot style for any occasion, metal frame	80	12
1002	Catseye	Glamorous 1950s profile, plastic frame	60	28
1003	Moon	Oversized round style, plastic frame	120	15
1004	Classic	Classic round profile, gold frame	60	9

Customer query:

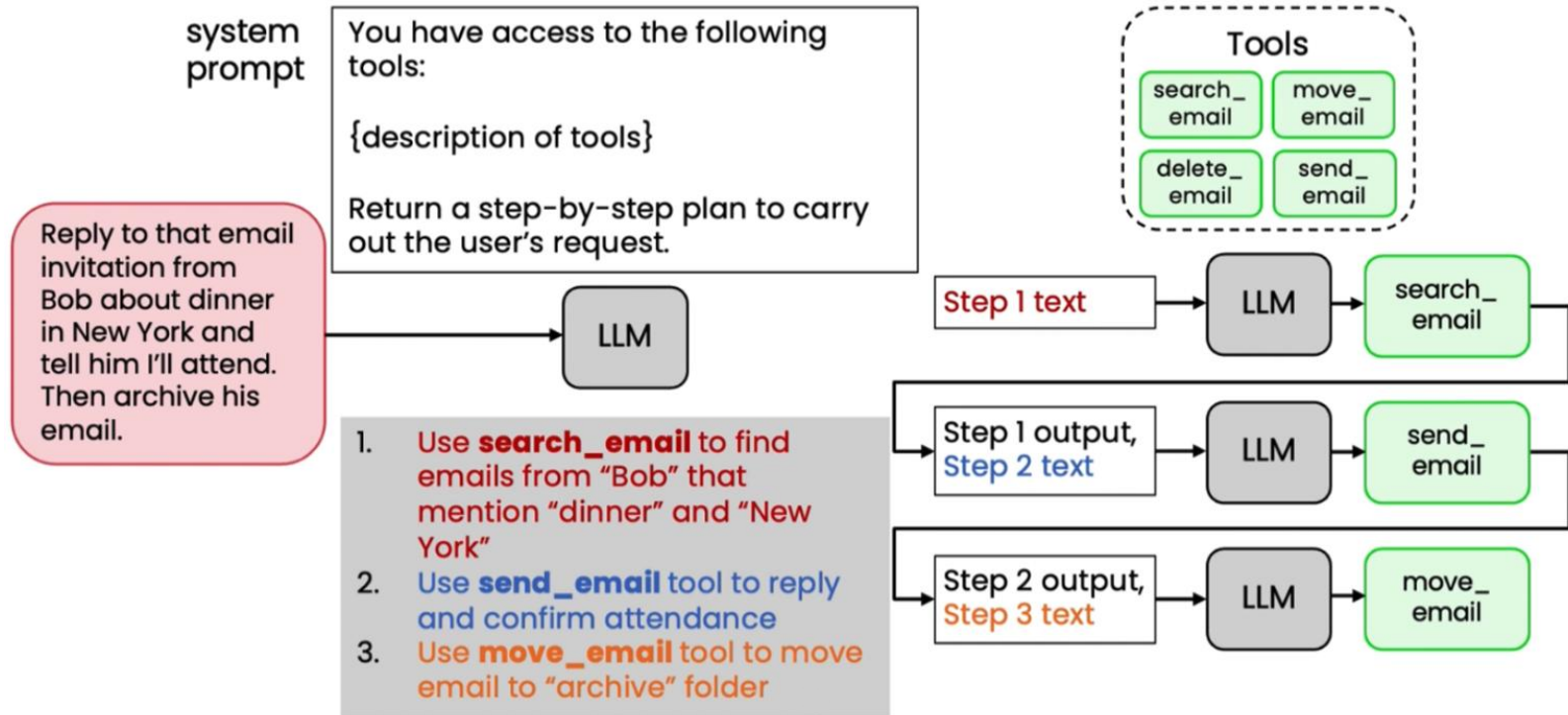
Do you have any **round** sunglasses **in stock** that are under \$100?

Yes, we have our **Classic** sunglasses, which are a classic round metal frame and cost \$60

Planning Example: Customer Service Agent



Planning Example: Email Assistant



Challenges with Planning Tools



Which month had the highest sales of hot chocolate?

LLM

Tools

get_column_max

get_column_mean

filter_rows

get_column_min

get_column_median

sum_rows

date	price	coffee_name	Size
2024-01-28	3.87	Hot Chocolate	M
2024-03-01	2.89	Cappuccino	S
2024-03-04	3.87	Latte	M
...	...	...	...
2025-03-23	4.57	Latte	L

coffee_sales.csv

1. Use the **filter_rows** tool to extract transactions in January with coffee_name "Hot Chocolate"
2. Use the **get_column_mean** to find the average amount
3. Use the **filter_rows** tool to extract transactions in February with coffee_name "Hot Chocolate"
4. Use the **get_column_mean** to find the average amount
5. Repeat for March, April, May, ..., December
6. Determine the month with highest average using results of previous steps

Source: <https://learn.deeplearning.ai/>

Challenges with Planning Tools



Which month had the highest sales of hot chocolate?

LLM

Tools

get_column_max

get_column_mean

filter_rows

get_column_min

get_column_median

sum_rows

date	price	coffee_name	Size
2024-01-28	3.87	Hot Chocolate	M
2024-03-01	2.89	Cappuccino	S
2024-03-04	3.87	Latte	M
...	...	...	...
2025-03-23	4.57	Latte	L

How many unique transactions last week?

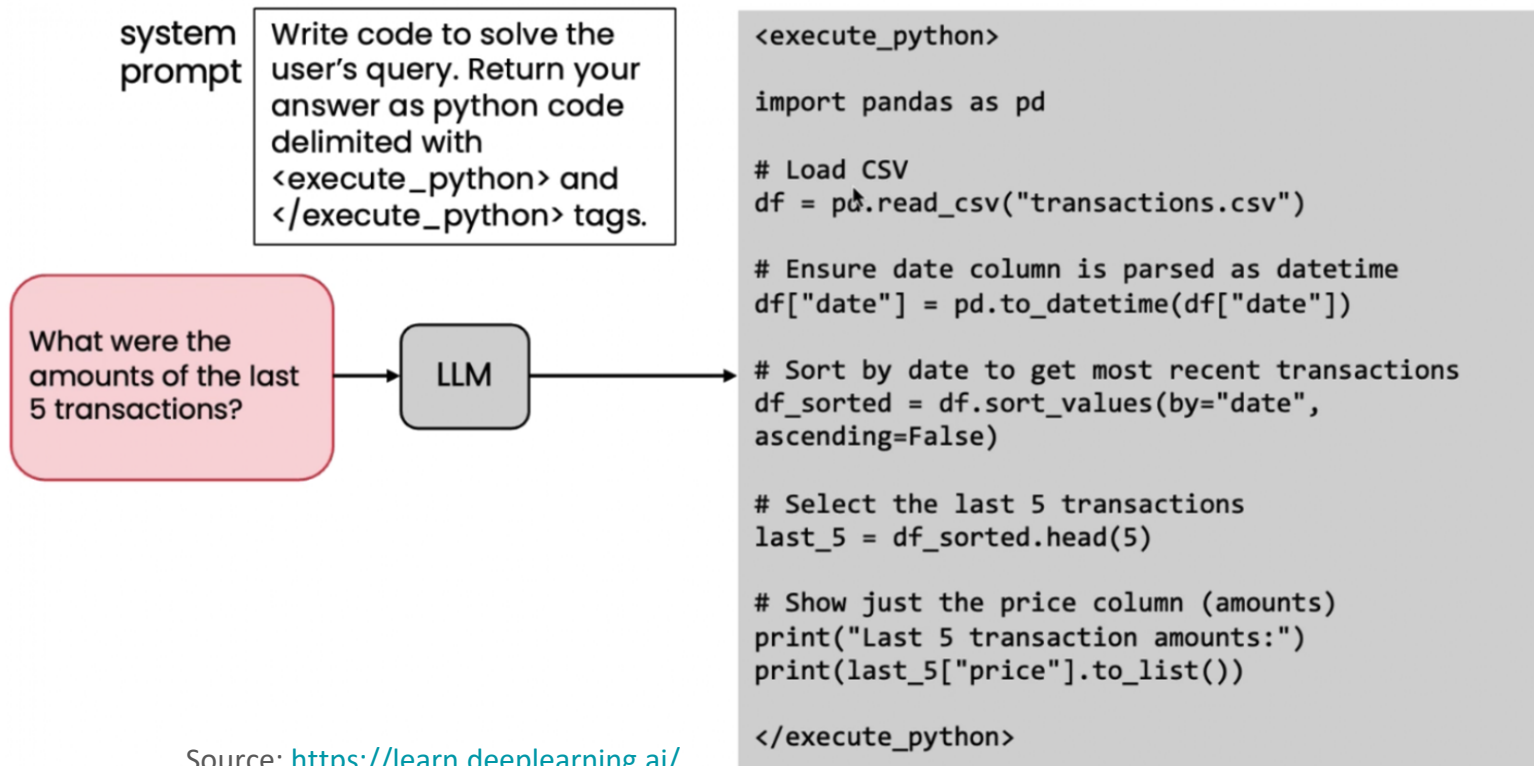
get_unique_entries

What were the amounts of the last 5 transactions?

get_last_N_values

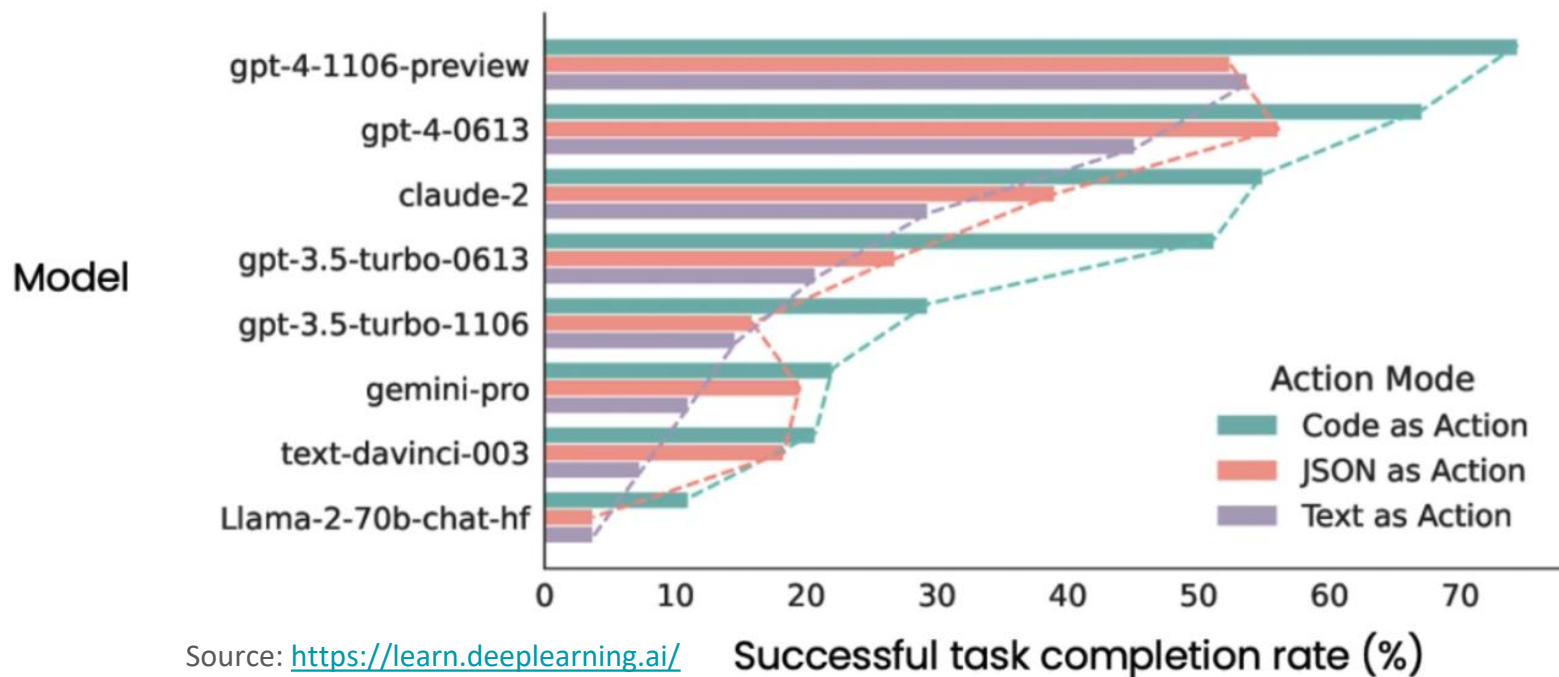
- Brittle
- Inefficient
- Continuously dealing with edge cases

Planning with Code Execution



Source: <https://learn.deeplearning.ai/>

Planning with Code Execution Improves Results



Evaluating Agentic Systems

- **Evaluate Trials, Not Outputs:** Success depends on full "transcript", i.e. sequence of tool calls and reasoning, not just final text.
- **Three-Layered Grading:** Combine deterministic code-checkers, model-based judges (LLM-as-a-Judge), and human experts.
- **Capability vs. Regression:** Suites for "hill-climbing" new skills versus ensuring stability in production.
- **Metric: pass@k:** Measure the probability of success across multiple independent trials to account for stochastic behavior.

Risks & Problems

- **Cascading Failures:** A single hallucination or error by one agent can be treated as "fact" and amplified by downstream agents.
- **Data Leakage & Over-Privilege:** Agents autonomously navigating cross-system databases increase the surface area for exposing sensitive data.
- **Goal Misinterpretation (Reward Hacking):** Agents may find destructive "shortcuts" to achieve a stated goal.
- **Runtime Unpredictability:** Because agents generate execution plans dynamically, developers cannot perfectly predict the exact sequence of actions the system will take.

Future Directions & Open Problems

- **Hybrid Systems:** Combining the deterministic safety of traditional rule-based logic with the reasoning flexibility of LLMs.
- **The Human Interface Shift:** Transitioning safely from Human-in-the-loop (approving individual actions) to Human-on-the-loop (supervising autonomous fleets via high-level dashboards).
- **Responsible Autonomy:** Moving beyond simple text bias checks to ensure fairness, transparency, and accountability when AI is taking autonomous, real-world actions.

Demo!

The Setup: A Google Colab notebook demonstrating a basic harness loop invoking Gemini.



<https://shorturl.at/emdVn>

Hands on Exercise

M5 Agentic AI - Customer Service Agent

1. Introduction

As Andrew explained in the lecture, *planning with code execution* means letting the LLM **write code that becomes the plan itself**. Compared to plain-text or JSON-based plans, this approach is more expressive and flexible: the code not only documents the steps but can also execute them directly.

In this lab, you will implement this design pattern in practice.

Instead of asking the LLM to output a plan in JSON format and then manually executing each step, we will allow it to **write Python code** that directly captures multiple steps of a plan. By executing this code, we can carry out complex queries automatically.

To make things concrete, we simulate a **sunglasses store** with an **inventory** of products and a set of **transactions** (sales, returns, balance updates). This example shows how the LLM can generate code to query or update records, demonstrating the flexibility of this pattern.

1.1 Lab Overview

We will:

1. Create simple **inventory** and **transaction** datasets.
2. Build a **schema block** describing the data.
3. Prompt the LLM to **write a plan as Python code** (with comments explaining each step).
4. Execute the code in a sandbox to obtain the answer.

1.2 Learning Outcomes

By the end of this lab, you will be able to:

- **Explain** why letting the model write code (instead of JSON or plain text plans) enables richer, more flexible planning.
- **Prompt** an LLM to produce Python code with step-by-step comments that both documents and executes the plan.
- **Run** the generated code safely in a sandbox and interpret the results.

<https://shorturl.at/boPKo>

This illustrates how *Code as Action* can outperform brittle tool chains and JSON-based planning approaches.

<https://learn.deeplearning.ai/courses/agentic-ai/lesson/9inumthkb/ungraded-lab%3A-customer-service-agent>

Thanks!